

PulseLedger

A correctness-first payment ledger that stays safe under concurrent transfers, request retries, and worker failures.

An achievable 8-week plan built around one strong idea: prove that financial records remain balanced and duplicate operations never move money twice.

8

weeks to a polished
v1

4

core correctness
invariants

1

source of truth:
PostgreSQL

0

unnecessary
infrastructure

Interview positioning

"I built a double-entry payment ledger that handles concurrent transfers and duplicate requests safely. PostgreSQL transactions protect the accounting core, an outbox worker preserves committed events, and property tests prove the invariants."

What PulseLedger solves

Payment systems operate on unreliable networks. Clients retry, requests overlap, processes stop, and workers restart. PulseLedger demonstrates how to keep the financial result correct through those conditions.

Without the project's safeguards

- A timed-out client retry charges twice.
- Concurrent withdrawals overspend an account.
- A debit is recorded without its matching credit.
- A committed transfer never reaches downstream processing.
- A repeated event creates a repeated business effect.
- Stored balance diverges from transaction history.

PulseLedger's response

- Double-entry postings conserve money.
- Serializable transactions handle concurrency.
- Idempotency returns one stable result.
- Transactional outbox preserves event intent.
- Consumer inbox rejects repeated effects.
- Reconciliation detects inconsistencies.

Success condition: a reviewer can clone the repository, start it locally, submit concurrent duplicate transfers, stop and restart the worker, and verify that the ledger remains balanced.

Primary use cases

USE CASE	EXPECTED BEHAVIOR	PROOF
Create an account	Account has fixed currency and immutable identity.	API and integration test.
Transfer value	Debit and credit commit atomically.	Transaction and property tests.
Retry a transfer	Original response returns; no second posting.	Concurrent duplicate test.
Recover worker	Committed outbox records are eventually handled.	Stop/restart integration test.
Verify balances	Journal-derived total matches account balance.	Reconciliation command.

Build the signal, remove the noise

Required for v1

- ☐ Account create/read API.
- ☐ Immutable double-entry journal.
- ☐ Serializable transfer transaction.
- ☐ Idempotency key and request fingerprint.
- ☐ Transactional database outbox.
- ☐ Simple idempotent audit worker.
- ☐ Balance reconciliation command.
- ☐ Structured logs and request IDs.
- ☐ Automated correctness/concurrency tests.
- ☐ Docker Compose and reproducible benchmark.

Removed from SDE-1 v1

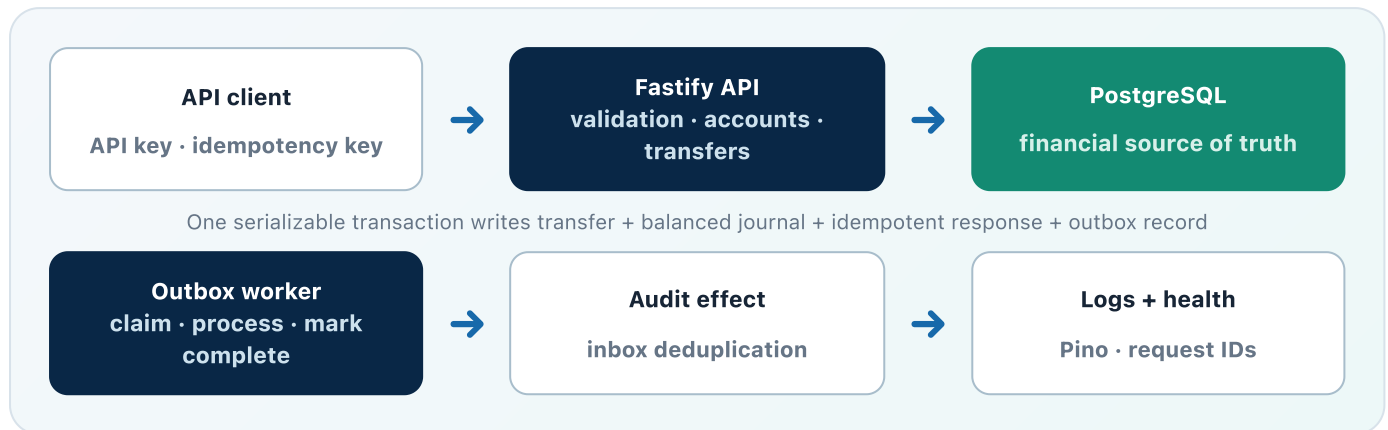
- Redpanda/Kafka broker.
- Redis and Lua rate limiter.
- Advanced multi-tenant control plane.
- HMAC signing and nonce replay store.
- Prometheus, Grafana and distributed tracing.
- Toxiproxy fault-injection infrastructure.
- Sophisticated DLQ/checkpoint replay system.
- Kubernetes or cloud deployment.
- Microservice decomposition.
- Customer-facing UI.

Scope decision

ORIGINAL IDEA	SDE-1 DECISION	REASON
Multi-tenant SaaS platform	Single platform; retain optional <code>tenant_id</code> column.	Shows data-model awareness without building administration.
Redpanda event streaming	Database-backed outbox worker first.	Proves the dual-write solution with much less setup.
Redis rate limiting	Post-v1 extension.	Does not strengthen the accounting story.
Full observability stack	Pino logs, request IDs, health routes.	Enough operational evidence for the initial project.
Complex security controls	Simple bearer API key or local-demo mode.	Keep effort focused on financial correctness.
Large failure matrix	Three high-value recovery tests.	Depth over infrastructure breadth.

Post-v1 rule: add Redpanda, Redis, tracing, or cloud deployment only after the release checklist is fully green and the addition has its own test and measurable purpose.

A modular monolith with one database



Core technology stack

LAYER	CHOICE	PURPOSE
Language/runtime	TypeScript + Node.js LTS	Type safety, async I/O, existing backend fluency.
HTTP framework	Fastify	Routes, validation, structured errors and testable plugins.
Database	PostgreSQL	ACID transactions, constraints, locks and query plans.
Database client	<code>pg</code> or <code>postgres.js</code>	Explicit SQL for transaction-critical paths.
Testing	Vitest + fast-check + Testcontainers	Unit, property, concurrency and real-database tests.
Logging	Pino	Structured request/error evidence.
Local environment	Docker Compose	One-command PostgreSQL and application startup.
Load testing	k6	Reproducible latency and concurrency measurements.
Automation	GitHub Actions	Lint, types, tests, migrations and build checks.

Why no broker in v1?

The transactional outbox is the important concept. A database worker is sufficient to demonstrate that committed work survives a process failure. A Kafka-compatible broker can be added later behind the worker boundary without changing ledger correctness.

Accounting truth before features

Minimal schema

TABLE	ESSENTIAL FIELDS	RULE
<code>accounts</code>	id, currency, status, created_at	Currency cannot change after creation.
<code>ledger_transactions</code>	id, reference, type, created_at	Groups balanced journal entries.
<code>journal_entries</code>	transaction_id, account_id, direction, amount_minor, currency	Positive integer amount; append only.
<code>idempotency_keys</code>	key, request_hash, status, response	Unique key; different payload causes conflict.
<code>outbox_events</code>	id, type, payload, attempts, processed_at	Written inside ledger transaction.
<code>consumer_inbox</code>	consumer, event_id, processed_at	Unique consumer + event ID.

Money representation

Use positive integer minor units and an explicit currency. ₹1,250.75 becomes `125075 INR`.
Never use floating point for stored money.

Balance authority

The immutable journal is authoritative. A stored balance may optimize reads, but reconciliation must recompute it from entries.

Minimal API

METHOD	ROUTE	PURPOSE
POST	<code>/v1/accounts</code>	Create an account.
GET	<code>/v1/accounts/:id</code>	Read account and balance.
GET	<code>/v1/accounts/:id/entries</code>	Read cursor-paginated journal.
POST	<code>/v1/transfers</code>	Transfer value; requires <code>Idempotency-Key</code> .
GET	<code>/v1/transfers/:id</code>	Read stable transfer result.
POST	<code>/v1/admin/reconcile</code>	Run journal/balance verification.
GET	<code>/health/live</code> , <code>/health/ready</code>	Process and database health.

Four invariants: debits equal credits; journal entries never mutate; one idempotency key has one meaning/result; each outbox event creates at most one logical consumer effect.

Weeks 1–4 · Build the ledger core

W1	Foundation TypeScript/Fastify repository, configuration validation, PostgreSQL Compose service, migrations, Vitest, Testcontainers, CI, health endpoints and account create/read routes.	Clean clone starts and creates an account.
W2	Double-entry journal Money value object, transaction/journal schema, balanced posting service, append-only policy, domain errors, unit tests and fast-check generated posting tests.	Valid postings always balance; invalid ones fail.
W3	Serializable transfer Source/destination validation, deterministic locking, atomic debit/credit, insufficient-funds rule, bounded serialization retries and transaction integration tests.	Concurrent transfers never violate balance rules.
W4	Idempotency Key table, request fingerprint, in-progress/completed state, stable response replay, same-key/different-body conflict and 50-request concurrency scenario.	50 identical concurrent requests create one transfer.

Week 4 checkpoint

- ☐ No floating-point money.
- ☐ Journal mutation is rejected.
- ☐ Debit equals credit property test passes.
- ☐ Insufficient funds cannot partially post.
- ☐ Retries use a bounded policy.

- ☐ Idempotency fingerprint conflict is tested.
- ☐ Concurrent duplicate test uses real PostgreSQL.
- ☐ Errors have stable machine codes.
- ☐ Logs contain request IDs.
- ☐ CI is green from clean checkout.

Hard gate: do not start the outbox worker while any ledger invariant or concurrent-idempotency test is failing.

Weeks 5–8 · Prove reliability and ship

W5	Transactional outbox Write event record with transfer, implement bounded batch claiming with <code>FOR UPDATE SKIP LOCKED</code> , attempts/backoff, processed timestamp and stop-after-commit recovery test.	Committed event survives worker interruption.
W6	Idempotent audit worker + reconciliation Inbox unique key, audit effect, duplicate event test, journal-derived balance computation, mismatch report and seeded-inconsistency test.	1,000 duplicates create one effect; mismatch is detected.
W7	Quality and benchmark Structured errors/logs, cursor pagination, realistic seed data, indexes with <code>EXPLAIN ANALYZE</code> , k6 normal/concurrent/duplicate/hot-account scenarios.	Benchmark is reproducible and invariants stay green.
W8	Release README, architecture diagram, four ADRs, API examples, trade-offs, clean-clone test, three-minute demo, tagged <code>v1.0</code> and final résumé bullet from real results.	Another developer reproduces setup and evidence.

Required architecture decisions

ADR-001

Why double-entry journal and integer minor units.

ADR-002

Why `SERIALIZABLE` plus bounded retry.

ADR-003

Why transactional outbox instead of dual write.

ADR-004

Why modular monolith and database worker for v1.

Feature freeze: Week 8 is for verification and communication. Do not add Redis, Kafka, dashboards, or cloud deployment during release week.

Tests that make the project credible

LAYER	REQUIRED PROOF	TOOL
Unit	Money validation, balanced postings, domain errors, retry policy.	Vitest
Property	Generated valid transfers preserve total value and never create invalid entries.	fast-check
Integration	Real constraints, transactions, serialization conflicts, outbox claiming and inbox uniqueness.	Vitest + Testcontainers
Concurrency	Duplicate requests and overlapping transfers cannot corrupt accounts.	Promise workers + PostgreSQL
Recovery	Worker stop/restart processes committed outbox record exactly once logically.	Integration harness
Load	Latency, throughput, retry rate and correctness under sustained scenarios.	k6

Mandatory scenarios

Accounting

- ☐ Debit equals credit.
- ☐ Negative/zero amount rejected.
- ☐ Currency mismatch rejected.
- ☐ Insufficient funds leaves no entries.
- ☐ Journal update/delete rejected.

Idempotency

- ☐ Same key/body returns original response.
- ☐ Same key/different body conflicts.
- ☐ 50 concurrent duplicates post once.
- ☐ Lost-response retry remains safe.
- ☐ Failed request policy is documented.

Concurrency

- ☐ Opposing transfers preserve totals.
- ☐ Hot account stays non-negative.
- ☐ Serialization retry budget is bounded.
- ☐ Deterministic lock order is used.
- ☐ Final state matches reference model.

Worker and recovery

- ☐ Committed outbox row survives stop.
- ☐ Two workers do not double-claim.
- ☐ Duplicate event effect is ignored.
- ☐ Failed attempt is visible.
- ☐ Reconciliation finds seeded mismatch.

Evidence rule: every résumé claim must point to a named test or committed benchmark. Line coverage alone is not proof of a financial invariant.

Make the result reproducible

Four benchmark scenarios

- Normal transfers at steady arrival rate.
- Concurrent transfers across many accounts.
- Duplicate storm using the same idempotency key.
- Hot-account contention to expose retry cost.

Record for every run

- Commit SHA and configuration.
- Hardware, operating system and container limits.
- Dataset size and access distribution.
- Duration, warm-up and load shape.
- Throughput, P50/P95/P99 and errors.
- Serialization conflicts and retry counts.

Repository layout

```
pulseledger/  
├── src/  
│   ├── modules/{accounts,ledger,idempotency,outbox,reconciliation}/  
│   ├── infrastructure/{database,logging}/  
│   ├── app.ts  
│   └── server.ts  
├── tests/{unit,property,integration,e2e}/  
├── benchmarks/k6/  
├── migrations/  
├── docs/{adr,diagrams}/  
├── docker-compose.yml  
└── README.md
```

README order

1. Problem and 60-second project summary.
2. Architecture and correctness invariants.
3. One-command local start and API examples.
4. How idempotency, isolation and outbox work.
5. Test commands and named failure scenarios.
6. Benchmark methodology and real results.
7. Trade-offs, limitations and post-v1 extensions.

No vanity metrics: keep TPS and latency fields blank until the committed k6 script produces them. Include raw output and exact hardware alongside the summary.

What “finished” means for SDE-1

Release checklist

- ☐ Fresh clone starts as documented.
- ☐ Four core invariants have named tests.
- ☐ Concurrent duplicate test passes.
- ☐ Worker recovery test passes.
- ☐ Reconciliation finds seeded mismatch.
- ☐ Benchmark can be reproduced.
- ☐ Architecture matches implementation.
- ☐ At least four ADRs are complete.
- ☐ Demo completes in under three minutes.
- ☐ `v1.0` tag points to verified commit.

Questions you must answer

- Why double-entry instead of two balance updates?
- Why store money in minor units?
- Why SERIALIZABLE, and what is its cost?
- How do concurrent idempotent requests work?
- What failure does the outbox solve?
- Why is delivery not truly exactly once?
- How does reconciliation detect corruption?
- What did the benchmark identify as bottleneck?
- Why did you avoid microservices?
- What would you add at 100× scale?

Three-minute demonstration

1. **0:00–0:30:** state the problem and four invariants.
2. **0:30–1:05:** create two accounts and complete a transfer.
3. **1:05–1:40:** send concurrent duplicate requests; show one posting.
4. **1:40–2:15:** stop and restart worker; show outbox recovery and one audit effect.
5. **2:15–2:40:** run reconciliation and show balanced result.
6. **2:40–3:00:** show measured benchmark and explain the bottleneck.

Résumé bullet—complete only after measurement

Built a double-entry payment ledger in TypeScript/Fastify/PostgreSQL with SERIALIZABLE transfers, concurrent-request idempotency and transactional outbox recovery; sustained **[measured throughput]** at **[measured P99]** on **[documented hardware]**, with **[duplicate count]** repeated requests producing one logical transfer.

Final principle: this is an SDE-1 project because it shows excellent fundamentals with honest boundaries—not because it contains every infrastructure technology.